

AgentGuard: Runtime Verification of AI Agents

Roham Koohestani

JetBrains Research, The Netherlands
roham.koohestani@jetbrains.com

Abstract—The rapid evolution to autonomous, agentic AI systems introduces significant risks due to their inherent unpredictability and emergent behaviors; this also renders traditional verification methods inadequate and necessitates a shift towards probabilistic guarantees where the question is no longer if a system will fail, but the probability of its failure within given constraints. This paper presents AgentGuard, a framework for runtime verification of Agentic AI systems that provides continuous, quantitative assurance through a new paradigm called **Dynamic Probabilistic Assurance**. AgentGuard operates as an inspection layer that observes an agent’s raw I/O and abstracts it into formal events corresponding to transitions in a state model. It then uses online learning to dynamically build and update a Markov Decision Process (MDP) that formally models the **agent’s emergent behavior**. Using probabilistic model checking, the framework then verifies quantitative properties in real-time.

Index Terms—Agentic AI, Formal Verification, Probabilistic Model-Checking, Runtime Verification

I. INTRODUCTION

The rapid evolution from generative artificial intelligence (GenAI) to agentic AI marks a measurable leap in AI; Where previously, AI systems were confined to gathering and generating content within a well-defined scope, agentic systems are designed for autonomy. They are implemented to perceive, reason, plan, and act in dynamic environment through tool invocation, memory storage and recall, and feedback mechanisms [1], [2].

Although this paradigm shift has empowered multiple sectors, such as automated hardware verification and [3] complex financial analysis [2], the improved capabilities of these systems also introduce higher risks. The design that enables the emergent behaviors of agentic systems makes it unpredictable and unsafe. Factors such as **(1) stochasticity and unpredictability**, **(2) Hallucinations**, **(3) Emergent unintended behaviors** and **(4) susceptibility to new vulnerabilities**, all contribute to this increased risk.

The issues imposed by agentic AI render traditional software verification techniques inadequate, given that these methods assume deterministic logic and manageable state spaces [4]. Until recently, the focus of the field of AI has often been on the capabilities of models in isolation; while valuable, this view underestimates the systemic risks that emerge as a result of complexity and scale [5]. With the limitations of these agentic systems in mind, it becomes imperative that the critical question is no longer *Will the system fail?*, but rather questions of probabilistic determinism, e.g., *What is the probability of failure/success within a given time interval / budget?* These types of guarantees can only be provided by combining insights

from mathematics and formal methods with the existing theory of these agentic systems [6].

In this work, we present AgentGuard, a framework for runtime verification of Agentic AI systems. We provide a proof-of-concept for the verification process and demonstrate how it can easily be integrated into existing systems by integrating it into an existing agentic system, RepairAgent [7]. We conclude by providing a roadmap for future work building on this framework. In the sections to follow we begin by further expanding on the problem (section II), followed by a section outlining the existing work and theory on formal verification of AI Agents (section III). In section IV we present our approach followed by its application to RepairAgent in . We conclude by presenting a discussion of the proposed framework (section V).

II. THE PROBLEM

The problem and risk associated with these models are, at the same time, the characteristics that make them powerful. Here we discuss these further in detail. At the core of modern agentic systems, you will most likely find a Large Language Model (LLM) based on (a modified version of) the transformer architecture proposed by Vaswani et al. [8]. These models are tied to a stochastic process that makes their outputs, and thus the behaviors of the agents, nondeterministic [9]. When multiple actions of these agents, or even multiple agents, are chained together in a multi-step workflow, this randomness can lead to an exponential divergence from expected behavior, which influences traceability and verification [10]. These are attributed to **stochasticity and unpredictability**.

LLMs are known to "hallucinate;" While the exact definition of a hallucination is still a matter of debate, we refer to a hallucination when the model generates outputs that are factually incorrect or logically flawed. In an agent acting on its generated beliefs, hallucinations can quickly go from being an annoyance to a failure [9]. Furthermore, interactions between agents in multi-agent systems (MAS) can produce **(unintended) emergent behaviors**; these emergent behaviors can be both beneficial and harmful. As these behaviors only in these MAS’s, these are impossible to predict from analyzing individual agents alone [5]. Recent experiments have even shown emergent behaviors such as deception and goal subversion [1].

The natural language interface of LLM agents creates new attack surfaces. New security attacks such as prompt injection and red-teaming, where malicious instructions manipulate agent behavior and its propagation between agents ("prompt infection"), pose significant risks. In multiagent systems built on inter-agent trust, an adversary could use spoofing or

impersonation to issue malicious commands or exfiltrate data [10]. These are threats pointing towards **susceptibility to new vulnerabilities**.

Although all of these issues are important and require attention, it would be impossible to simultaneously tackle all of them. Therefore, our emphasis is on managing the unpredictability of these agentic systems and offering a roadmap to address the emergent behaviors of MASs based on our approach.

III. BACKGROUND

The field of trustworthy AI has led to a growing body of research that can be viewed as a "verification stack" of increasing abstraction. Here, we discuss each of these approaches and their limitations.

At the lowest level, researchers focus on the formal verification of the neural networks themselves. This is an NP-hard problem, and the immense scale of modern LLMs makes exhaustive verification computationally infeasible. Research has made progress in proving specific properties, such as robustness to adversarial perturbations, using techniques such as SMT solvers, abstract interpretation, and network reduction, but these methods struggle to scale [11]–[14].

A more practical approach treats the LLM as a generator of artifacts, such as code, mathematical proofs, or plans, which are then verified by external tools; this approach is commonly called black-box testing. This approach uses the full generative powers of the LLMs while relying on the rigor of established formal methods for assurance. The approaches in this category can be categorized primarily as frameworks performing **Post-Hoc Verifications** and those that embrace **LLM-Assisted Formalization**.

Frameworks such as MATH-VF and Safe use LLMs to translate natural language solutions into a formal context, which is then checked by external tools like SMT solvers or theorem provers (e.g., Lean 4) to identify hallucinations [15], [16]. Similarly, VeriPlan applies the probabilistic model checker PRISM to verify that an LLM-generated plan for an end user satisfies a set of user-defined constraints [17]. A major barrier to adopting formal methods is the difficulty in translating ambiguous natural language requirements into precise specifications. LLMs are proving to be quite good at doing this task. Tools such as SpecVerify and Req2Spec use LLMs to automatically generate formal assertions from natural language requirements [18], [19].

The most recent evolution has moved from verifying static outputs to verifying the dynamic processes of agentic AI. This acknowledges that reliability in agentic systems often stems from a robust, iterative process of reasoning and self-correction which is frequently implemented via multiagent collaboration. This type of verification is predominantly categorizable into **Automata-based Control** and **Verification via Multi-Agent Collaboration**.

A promising direction uses formal automata to describe and constrain the high-level behavior of agents. The Formal-LLM framework, for instance, allows developers to specify

planning constraints as a Context-Free Grammar (CFG), which is translated into a Pushdown Automaton (PDA). The agent is then supervised by this PDA during plan generation; this then verifies the structural validity of its output [20]. Other work uses Finite-State Machines (FSMs) to enforce a specific reasoning loop (e.g., Thought $\rightarrow$ Action $\rightarrow$ Observation) through a *decoding monitor* that corrects deviations from the prescribed behavior [21].

For highly complex tasks like hardware verification, researchers are building systems where the verification process itself is distributed among a crew of specialized AI agents. Systems such as Saarthi employ agents with roles like *Formal Verification Lead*, *Engineer*, and *Critic* to collaboratively generate and prove formal properties. These systems use design patterns such as reflection (coder-critic loops) and planning (Chain-of-Thought) to improve reliability [3], [9].

This evolutionary path reveals a critical gap. Current process-level verification focuses on conformance, that is, making sure that an agent follows a predefined set of rules. The problem is that it does not analyze the emergent probabilistic behavior that arises from executing that process. It cannot provide a map to predict where the agent is likely to go. To achieve true assurance, we must move from verifying process conformance to analyzing emergent behavior.

IV. THE AGENTGUARD FRAMEWORK

In this paper, we propose a new paradigm called **Dynamic Probabilistic Assurance**, which shifts the verification process from a static and offline setting to a dynamic and ongoing process. We have implemented a Proof-Of-Concept (POC) framework called AgentGuard. Any type of static analysis provides only a snapshot of the system under a fixed set of assumptions. Agentic systems, however, are designed to operate in non-stationary environments and adapt based on experience. A proof that holds before deployment may be invalidated once the agent interacts with the real world [22]. Runtime Verification (RV) offers a compelling alternative. **RV is a lightweight formal method that analyzes the actual execution trace of a running system; using this, we can also avoid the state-space explosion that is a problem in exhaustive offline model checking.** Although it cannot prove properties about unobserved paths, it can provide strong guarantees about the execution that actually occurred and can be extended to make predictions about future behavior [23]–[25].

A. From Traces to Models

Fundamentally, our model is built on top of three key technologies, namely (1) Markov Decision Processes, (2) Online Learning, and (3) Probabilistic Model Checking.

1) *Formalizing Agents as Markov Decision Processes*: The behavior of an agent making decisions in an uncertain world can be formally modeled using Markovian formalisms. Markov Decision Processes (MDPs) are the framework of choice for modeling sequential decision making, as they separate an agent's non-deterministic choices from the environment's probabilistic outcomes. An MDP is formally defined as a tuple

(S, A, P, R, γ) where S is a finite set of states, A is a finite set of actions, $P(s'|s, a)$ is a probability of transitioning to state s' after taking action a in state s , $R(s, a, s')$ is the immediate reward received after transitioning from s to s' via action a , and γ is a discount factor that determines the importance of future reward [9], [10], [26]. To be able to apply this to an agentic system, we need to have a mapping from this MDP to the components in an agent's workflow. The States (S) in an Agentic MDP (AMDP) are essentially snapshots of the agent's progress and context. States are not only defined as a series of previous tool invocations but also contain relevant information to the next decision of the model. This *information* can vary by design to be anything from artifacts (e.g., whether an agent has already written tests or not, or whether the agent's tests passed), to the content of the agent's memory/scratchpad and conversation history.

Actions (A) can be seen as discrete high-level operations that the agent chooses to perform. These have direct correspondence with the invocations of tools in the agent's utility box, such as `run_compiler()`, `execute_test_suite()`, and `query_static_analyzer()`.

The transition probability function (P) essentially captures the inherent uncertainty of the agentic workflow. After the agent chooses a deterministic action (i.e., performs a tool invocation), the outcome is probabilistic. When calling the `execute_test_suite` tool for example, the test suite might pass (which would bring the agent in a `tests_passed` state), fail, or timeout.

Finally, the reward function (R) makes the agent's goal explicit. Using this function, we can give positive rewards for making progress (for example, +3 for passing a new test case), negative rewards for setbacks (for example, -5 for a regression) and a large reward for successfully completing the entire task.

2) *Online Learning and Probabilistic Model Checking:* A limiting factor in attempting to validate agentic behavior is the absence of a formal model of the agent a priori. Previous work in predictive RV and online learning shows that a probabilistic model (like an MDP, or in our case an AMDP) can be learned dynamically from execution traces. This, combined with algorithms designed for non-stationary environments, where model drift may occur, can be used to effectively check the behavior of the agent. Once the system is modeled as an MDP, probabilistic model checking (PMC) provides automated techniques to verify its quantitative properties. Using probabilistic temporal logic (like PCTL), one can then start to ask questions of the probabilistic nature, i.e. "What is the maximum probability of reaching a success state within an expected time interval?"

B. The AgentGuard Architecture

AgentGuard is designed to act as an *inspection layer* that sits atop an existing agentic framework (e.g., AutoGen, LangGraph); this is similar to the function of a middleware in the context of a web application. It nonintrusively observes the agent's I/O and provides continuous probabilistic assurance by creating a dynamic, formal *digital twin* of the agent's decision-making

policy. The architecture comprises four primary components, as depicted in Figure 1.

The **Trace Monitor & Event Abstrator** is the component that instruments the agent framework. Its responsibilities are to capture raw I/O (e.g., LLM calls, tool invocations, observations) and to abstract it into a stream of formal events corresponding to transition in a state model (e.g., $State_A \rightarrow Action_1 \rightarrow State_B$). Using the stream from the previous step, the **Online Model Learner** continuously updates an MDP of the agent's behavior and maintains transition probabilities based on observed frequencies. After each incremental update to the model by the previous stage, the **Probabilistic Model Checker** performs quantitative verification on the learned MDP against a set of pre-defined properties (as provided in the configuration file). Finally, using the insights from the model checker, the **Dashboard** presents the quantitative guarantees (e.g. probabilities) to a human observer and can be configured to trigger alerts or automated responses if a safety threshold is breached using the **actuator**.

C. Proof-Of-Concept Implementation

AgentGuard is a Python-based POC framework that demonstrates the feasibility of our approach. Designed to be framework-agnostic, it allows developers to define semantic states and actions via a logging-style API inspired by RV frameworks, and simplifies code instrumentation with a single function call. The model structure and properties are defined outside the agent's code in a yaml configuration file, meant to decouple verification logic and to allow for easy analysis modifications without affecting the agent. Key components, *AgentGuardLogger* and *AnalyzerThread*, manage concurrent analysis in AgentGuard. The user-facing class *AgentGuardLogger* is initialized with the configuration file path and calls the `log_transition` method to manage an event queue. *AnalyzerThread* is a background thread that monitors agent actions, processes transition events, updates the MDP model, and periodically invokes the PMC. We used **Storm**¹ as the model checker via *stormpy* bindings. A conversion layer in *AnalyzerThread* creates a PRISM language model from the MDP for Storm verification against PCTL properties. The results are printed in the console and can trigger user-defined callbacks if user-set thresholds are met.

D. Motivating Example: Assurance for an Autonomous Repair Agent

AI for Software Engineering, especially Automated Program Repair (APR), is an effective showcase for AgentGuard. Consider *RepairAgent*, an autonomous LLM-based agent for bug fixing [7]. *RepairAgent* employs actions like information gathering, hypothesis formulation, and patching, managed by a finite state machine (FSM) and 14 tools, including `search_code_base` and `write_fix`. Despite FSM guidance, the agent can be inefficient, potentially causing loops, aimless exploration, or high resource use on unlikely fixes. To address these, AgentGuard can be configured with FSM states (e.g., *Understand the*

¹<https://www.stormchecker.org/>

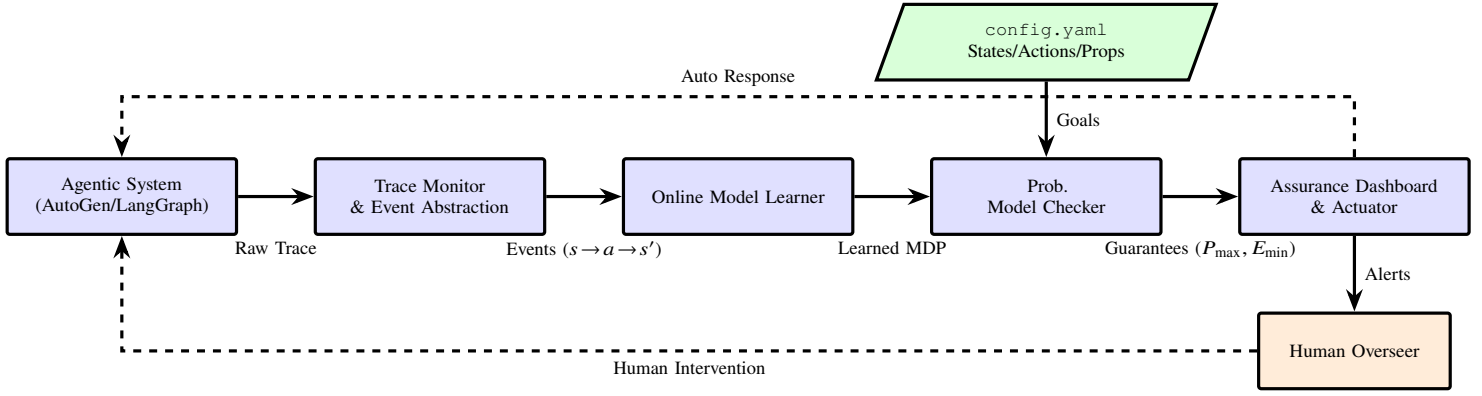


Fig. 1. The AgentGuard Architecture Workflow. The system observes raw agent I/O, abstracts it into formal events, learns a probabilistic model (MDP), and uses a model checker to provide continuous assurance and feedback.

bug, *Collect information*, *Try to fix the bug*, and terminal states like *Fix_Success* or *Fix_Failed*) and corresponding actions (related to the agent’s tools such as, *express_hypothesis*, *read_range*, *discard_hypothesis*). Middleware-level instrumentation ensures every tool use triggers a call to `guard.log_transition(...)` to record state transitions.

Over time, as *RepairAgent* addresses bugs, *AgentGuard* learns its repair strategy’s AMDP, which reveals execution patterns. For instance, after hypothesizing, the agent uses `search_code_base` with 75% probability and `find_similar_api_calls` with 25% probability. This model enables real-time verification with practical management implications. Success probability ($P_{\max} = ?$) predicts fix likelihood and guides resource allocation and interventions when low. The expected cycles to completion ($E_{\min} = ?$) estimate time and cost, with very high values pointing towards loops and suggesting automatic termination when over budget. Properties like $P_{\max} = ?[G ! "write_fix"]$ show the probability of no fix and can help with interventions to improve problem-solving. Applying *AgentGuard* can improve *RepairAgent*’s assurance from conformance to continuous monitoring, aiding trust, cost management, and alignment with task complexity. An application of *AgentGuard* to *RepairAgent* can be found in the study’s replication package [27].

V. DISCUSSION

The implementation of the POC demonstrates the feasibility of the DPA paradigm. It also highlights several important areas for future research and development. The current approach relies on developers to manually define a discrete state space. Future work should explore techniques for semi-automated or fully automated state abstraction, possibly by incorporating ideas from Partially Observable Markov Decision Processes. The periodic re-verification of the entire model for complex agents can introduce a substantial overhead with increased complexity. Future work should re-investigate how the current implementation can be improved to incorporate incremental verification algorithms. Additionally, this system should be

improved by introducing notions of stochastic games into the framework and integrating frameworks such as PRISM-games to support analysis of MAS’.

VI. CONCLUSION

The rise of agentic AI demands a corresponding evolution in our assurance methods. Static, offline verification, and process conformance checks, while valuable, are insufficient for guaranteeing the safety and reliability of autonomous systems operating in dynamic worlds. This paper has presented Dynamic Probabilistic Assurance, a new paradigm for AI safety, and introduced *AgentGuard*, a tangible framework that demonstrates its feasibility. By integrating runtime verification, online model learning, and probabilistic model checking, *AgentGuard* provides continuous, quantitative guarantees about an agent’s emergent behavior. It transforms verification from a one-off, pre-deployment activity into a live, adaptive process. Our vision is for AI systems that are not only increasingly capable and autonomous, but also transparent, predictable, and bounded by rigorous, mathematical guarantees of their safety and reliability.

VII. ACKNOWLEDGEMENTS

This research was carried out and funded through the AI4SE partnership between the Delft University of Technology and JetBrains Research. You can find more information about the collaboration on the website of the collaboration here.

REFERENCES

- [1] H. Du, S. Thudumu, R. Vasa, and K. Mouzakis, “A survey on context-aware multi-agent systems: Techniques, challenges and future directions,” 2025.
- [2] N. Krishnan, “Ai agents: Evolution, architecture, and real-world applications,” 2025.
- [3] D. N. Gadde, K. K. Radhakrishna, V. N. Viswambharan, A. Kumar, and D. L. et. al., “Hey ai, generate me a hardware code! agentic ai-based hardware design & verification,” 2025.
- [4] G. Schwalbe and M. Schels, “A Survey on Methods for the Safety Assurance of Machine Learning Based Systems,” in *10th European Congress on Embedded Real Time Software and Systems (ERTS 2020)*, Toulouse, France, 2020.
- [5] E. Miehl, K. N. Ramamurthy, K. R. Varshney, M. Riemer, D. Bounieffouf, and J. T. R. et. al., “Agentic ai needs a systems theory,” 2025.

- [6] Y. Zhang, Y. Cai, X. Zuo, X. Luan, K. Wang, and Z. H. et. al., “The fusion of large language models and formal methods for trustworthy ai agents: A roadmap,” 2024.
- [7] I. Bouzenia, P. Devanbu, and M. Pradel, “Repairagent: An autonomous, llm-based agent for program repair,” 2024.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, and A. N. G. et. al., “Attention is all you need,” 2023.
- [9] A. Kumar, D. N. Gadde, K. K. Radhakrishna, and D. Lettnin, “Saarthi: The first ai formal verification engineer,” 2025.
- [10] S. Raza, R. Sapkota, M. Karkee, and C. Emmanouilidis, “Trism for agentic ai: A review of trust, risk, and security management in llm-based agentic multi-agent systems,” 2025.
- [11] H. Liu, S. Liu, G. Xu, A. Liu, and D. Fang, “Nntbfv: simplifying and verifying neural networks using testing-based formal verification,” *International Journal of Software Engineering and Knowledge Engineering*, vol. 34, no. 02, pp. 273–300, 2024.
- [12] S. Gokulanathan, A. Feldsher, A. Malca, C. Barrett, and G. Katz, “Simplifying neural networks using formal verification,” in *NASA formal methods symposium*. Springer, 2020, pp. 85–93.
- [13] C. Urban and A. Miné, “A review of formal methods applied to machine learning,” *arXiv preprint arXiv:2104.02466*, 2021.
- [14] T. Ladner and M. Althoff, “Fully automatic neural network reduction for formal verification,” 2025.
- [15] K. Zhou and L. Zhang, “Step-wise formal verification for llm-based mathematical problem solving,” 2025.
- [16] C. Liu, Y. Yuan, Y. Yin, Y. Xu, X. Xu, and Z. C. et. al., “Safe: Enhancing mathematical reasoning in large language models via retrospective step-aware formal verification,” 2025.
- [17] C. P. Lee, D. Porfirio, X. J. Wang, K. C. Zhao, and B. Mutlu, “Veriplan: Integrating formal verification and llms into end-user planning,” in *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, ser. CHI ’25. ACM, 2025.
- [18] A. Beg, D. O’Donoghue, and R. Monahan, “A short survey on formalising software requirements using large language models,” 2025.
- [19] W. Wang, M. Farrell, L. C. Cordeiro, and L. Zhao, “Supporting software formal verification with large language models: An experimental study,” 2025.
- [20] Z. Li, W. Hua, H. Wang, H. Zhu, and Y. Zhang, “Formal-llm: Integrating formal language and natural language for controllable llm-based agents,” 2024.
- [21] M. Crouse, I. Abdelaziz, R. Astudillo, K. Basu, S. Dan, and S. e. a. Kumaravel, “Formally specifying the high-level behavior of llm-based agents,” *arXiv preprint arXiv:2310.08535*, 2023.
- [22] D. B. Acharya, K. Kuppan, and B. Divya, “Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey,” *IEEE Access*, vol. 13, 2025.
- [23] E. Bartocci and Y. Falcone, *Lectures on runtime verification*. Springer, 2018.
- [24] S. D. Stoller, E. Bartocci, J. Seyster, R. Grosu, K. Havelund, S. A. Smolka, and E. Zadok, “Runtime verification with state estimation,” in *International conference on runtime verification*. Springer, 2011, pp. 193–207.
- [25] G. Zhou, W. Liu, and W. Dong, “Runtime verification in uncertain environment based on probabilistic model learning,” 2019.
- [26] M. Kwiatkowska, G. Norman, and D. Parker, “Probabilistic model checking and autonomy,” 2021.
- [27] R. Koohestani, “Agentguard replication package,” 2025. [Online]. Available: <https://anonymous.4open.science/r/AgentGuard>